

ACPollerForAPS

Invoice pipeline

Routing, mapping and delivery of APS invoice exports to ERP formats

Product Manual

Version 2.0

Arondor

Activate your content

Contents

1. Introduction & concepts
2. Installation & startup
3. The configuration interface
4. Configuring a channel
5. Field mapping (incl. conditional Vendor/Ledger)
6. Output transports (FS / FTPS / S3) & delivery retry
7. ERP Providers (plugins) — in depth
8. Operations: logs, monitoring, lifecycle, troubleshooting

1. Introduction & concepts

ACPollerForAPS converts invoices exported from APS (ReadSoft) into the format each destination ERP expects, and delivers them where the ERP imports them. A single input format (APS XML), multiple outputs depending on the Buyer: CSV for Dynamics AX, XML for Optima, and so on — all configuration-driven, with no recompilation.

The pipeline

At a scheduled interval, the service:

1. **Scans** the input folder for APS XML files.
2. **Routes** each file to an output channel, based on its Buyer value.
3. **Merges** the files of the same channel (optionally in batches).
4. **Converts** to the channel's format via its provider.
5. **Delivers** through the channel transport (FS, FTPS or S3).
6. **Archives** the source files — only if delivery succeeded.

Architecture

Component	Role
ACPollerForAPS.Core (DLL)	Shared model, mapping engine, validation, encryption, path extractor, plugin contract. Referenced by both the service and the UI — single source of truth.
ACPollerForAPS.Service.exe	The Windows service that runs the pipeline.
ACPollerForAPS.UI.exe	The configuration interface (WPF).

The configuration file

Both the service and the UI share a single `settings.json` (structure `{ "Pipeline": { ... } }`). The service loads it at startup, next to its exe; the UI loads it automatically at launch if present, and edits it.

The service reads `settings.json` once, at startup. After a change through the UI, **restart the service** so it picks up the new configuration.

2. Installation & startup

Prerequisites

- .NET Framework 4.8 (runtime) on the target machine.
- Administrator rights (Windows service + Event Log).

Installer

Run the setup (`ACPollerForAPS-Setup-2.0.0.exe`). It asks for the install directory and lets you choose components: the **Service** and the **Configuration interface**. The installer stops a running service/UI first, copies the files, installs the service (which also creates the Windows Event Log source) and starts it. A default `settings.json` is deployed only if none exists, so the service can start on a fresh machine.

The uninstaller stops the service and the UI, removes the binaries and the service, but keeps `logs\` , `providers\` and `settings.json` so configuration and history survive an upgrade.

3. The configuration interface

The window has a branded header, a toolbar (New, Open, Save, Save as, Validate on the left; Dark theme, Help, Manual on the right) and three tabs: General, Schedule, Output channels.

General tab

Field	Meaning
Input folder	Folder the service watches for incoming APS XML files.
Archive folder	Where processed files are moved (dated) after successful delivery.
Error folder	Where unroutable / unreadable files are moved for review.
Archiving enabled	If off, sources are deleted instead of archived.
File filter	Which files to pick up (e.g. <code>*.xml</code>).
Stability check (ms)	Delay to confirm a file is fully written before processing.
Record path	XPath to a single invoice, e.g. <code>/InvoiceData</code> .
Buyer path	XPath (relative to the record) to the value that selects the output channel.

Schedule tab

Run interval (minutes or hours). Each run processes everything present.

A 24h interval means one run per day; for testing, set a few minutes.

Output channels tab

The channel list is on the left; the selected channel's editor is on the right, with its settings, transport, field mapping and preview.

4. Configuring a channel

A channel describes one ERP destination: which Buyers it serves, the output format, where and how it is delivered, and how invoices are mapped.

Field	Meaning
Channel name / Enabled	A label; a disabled channel is ignored.
Buyers (semicolon)	Buyer values routed to this channel.
Output format	Csv or Xml.
Output folder / file name	Destination and name. Tokens: <code>{date}</code> <code>{time}</code> <code>{guid}</code> <code>{batch}</code> .
Record path / Lines path	Invoice and repeating accounting lines location.
Provider	<code>mapping</code> (default) or a plugin DLL name — see section 7.
Sample XML	A representative input XML, stored in the config; feeds preview and Path auto-completion.
Invoices per file	Max invoices per output file. <code>0</code> = one merged file. E.g. 5 with 30 invoices ⇒ 6 files of 5.

5. Field mapping

The mapping grid defines the output columns (CSV) or elements (XML), in order.

Property	Meaning
Name	Output column / element name.
Type	<code>text</code> · <code>amount</code> (decimals; Abs drops the sign) · <code>date</code> (Date in = input format).
Source	<code>fixed</code> = constant (Value) · <code>header</code> / <code>xpath</code> = invoice node (Path) · <code>line</code> = current accounting-line node (Path).
Value / Path	Constant (fixed) / XPath (others). Path suggestions are filtered by Source.
Dec. / Abs	Decimals / absolute value for amounts.
Date in	Input date format.
Only when (path) / equals	Conditional write — see below.

Conditional mapping (Vendor / Ledger lines)

Some formats (Dynamics AX) produce, per invoice, one **Vendor** line (vendor credit) and **N Ledger** lines (expense debits), where some columns differ depending on the line type. A field is written only when a node of the line equals a value; otherwise the cell is empty.

Column	Only when (path)	equals
Credit	PostingType	Credit
Debit	PostingType	Debit
Vendor control account	PostingType	Credit

This turns a single invoice (one Credit line + several Debit lines in the XML) into one Vendor row and several Ledger rows in the CSV, each with the right columns.

Preview

Load a Sample XML in the channel, then click **Preview** to see the generated output. **Refresh paths** re-scans the sample XML. The **Validate** button (toolbar) checks the whole configuration and reports errors and warnings before saving.

6. Output transports

Each channel delivers its file through a transport, with a configurable retry (count + delay).

FS — file system

The file is written to the channel's Output folder.

FTPS

Field	Meaning
Host / Port	Server; 21 (explicit) or 990 (implicit).
Remote folder	Target folder (created if missing).
Username / Password	Password stored encrypted (leave empty to keep current).
FTPS mode	Explicit (FTPES) or Implicit.
Validate certificate	Verify the TLS certificate (off only for test servers).

S3 (and S3-compatible / MinIO)

Field	Meaning
Bucket / Key prefix	Target bucket; optional logical folder.
Region / Endpoint	AWS region, or a custom endpoint for MinIO / S3-compatible.
Path-style access	Required by MinIO.
Access key / Secret key	Secret stored encrypted.

Credentials are encrypted with DPAPI (machine scope). Encrypt on the target machine (the one running the service); a secret encrypted elsewhere cannot be decrypted there.

Failed deliveries & automatic retry

When a delivery fails after all retries (FTPS/S3 unreachable), nothing is lost:

- The generated output file is moved to a **pending queue** (`pending/<channel>/`), with a companion file recording its target channel.
- The source XML files are archived normally — the generated content is safe.
- At the start of every run, before new files, the service retries the queued deliveries. Files that go through leave the queue; the others stay.

A failed delivery is therefore retried automatically and completed — the exact file that was generated — as soon as the server is reachable again, in chronological order. Pending deliveries appear in the run summary and raise a warning in the Windows Event Log.

7. ERP Providers (plugins) advanced

The provider mechanism makes the product extensible to any ERP without touching the core. Hybrid approach:

Level	When	How
Mapping (default)	Most ERPs — output describable by the field mapping.	No DLL. <code>Provider = mapping</code> + configure the grid.
DLL plugin	An ERP with specific logic the mapping cannot express.	Ship a DLL implementing <code>IErpExporter</code> in <code>providers/</code> ; set <code>Provider</code> to its name.

The contract

```
public interface IErpExporter
{
    IEnumerable<string> ProviderNames { get; }
    ExportResult Export(IEnumerable<string> inputXmIs, OutputChannel channel);
}
```

Building a provider

New class library (.NET 4.8) referencing `ACPollerForAPS.Core` → implement `IErpExporter` → fill `result.Content` → compile → drop the DLL in `providers/` next to the service exe → set the channel `Provider` to its name. No service recompilation. The service discovers it at startup; a broken DLL is logged and ignored.

Version compatibility: a provider is compiled against a version of the Core DLL. The Core shipped next to the service must be compatible; recompile providers if the contract changes.

8. Operations

Logs

NLog writes to `logs/` next to the service exe, with daily + size rotation and automatic purge. Each run logs a start line, per-file detail and a summary. Set the level to `Debug` in `NLog.config` for diagnostics, then back to `Info`.

Monitoring (Windows Event Log)

The service reports to a dedicated Windows event log **ACPollerForAPS** (Event Viewer → Applications and Services Logs). Business events plus warnings and errors, graded by severity. Any enterprise monitor (Nagios, Zabbix, Centreon) reads it natively.

ID	Severity	Meaning
1000 / 1001	Information	Service started / stopped
1100	Information	Run completed OK (summary)
1200	Warning	Run completed with anomalies
1300	Error	Delivery failure / files still pending

PowerShell: `Get-EventLog -LogName ACPollerForAPS -Newest 20`. The event source is created at service installation.

Service lifecycle

```
sc start ACPollerForAPS
sc stop ACPollerForAPS
```

Restart the service after changing `settings.json`.

Troubleshooting

Symptom	Likely cause
Service installs but won't start (error 1064)	Missing <code>settings.json</code> next to the exe, or an invalid configuration. Check <code>logs\</code> and the Event Log.
Service runs but processes nothing	Interval too long (waits for its slot), wrong input folder, or no channel matches the Buyer.
InstallUtil returns -1	Leftover Event Log source, or a missing dependency DLL next to the service exe.
FTPS/S3 secret cannot be decrypted	Encrypted on a machine other than the service's (DPAPI machine scope).
Files stuck in <code>pending/</code>	Delivery keeps failing (server unreachable, bad credentials). Files are safe and retried each run; check the transport settings and the Event Log.